

GUIA COMPLETO DA AULA

Pipeline ELT com Snowflake + dbt Core + Airflow

TPC-H | RBAC | staging | marts | macros | testes | Cosmos

Objetivo - Acompanhar o professor passo a passo sem precisar voltar na transcrição. O guia segue a ordem da aula e separa o que aparece na aula do que é apenas uma observação prática para o seu ambiente Windows.

Baseado na transcrição fornecida da aula. Versão de acompanhamento personalizada para o projeto data_pipeline.

Mapa da aula

- 1. Conceito: ETL x ELT
- 2. Arquitetura e ferramentas
- 3. Preparação do Snowflake: warehouse, database, role, grants e schema
- 4. Inicialização do dbt Core e conexão com Snowflake
- 5. Estrutura do projeto dbt e dbt_project.yml
- 6. Pacote dbt_utils e dbt deps
- 7. Sources do TPC-H e testes de origem
- 8. Staging de ORDERS
- 9. Staging de LINEITEM e surrogate key
- 10. Modelos intermediários e conceito de fact table
- 11. Macro de pricing/desconto
- 12. Summary + fct_orders
- 13. Testes genéricos e singulares
- 14. Airflow + Astronomer Cosmos
- 15. Snowflake Connection no Airflow
- 16. Execução do DAG, Graph e logs
- 17. Checklist final e comandos para memorizar

Legenda - Blocos "AULA" seguem a lógica descrita na transcrição. Blocos "NO SEU PROJETO" usam os nomes que você já configurou: DBT_WH, DBT_DB, DBT_ROLE, DBT_SCHEMA e projeto data_pipeline. Quando a transcrição não preserva um código que estava visível na tela do professor, isso é indicado explicitamente.

1. ETL x ELT

A aula começa explicando por que o fluxo moderno usado no projeto é ELT e não ETL.

Fluxo	Ordem	Ideia central
ETL	Extract -> Transform -> Load	Transformar antes de carregar no warehouse.
ELT	Extract -> Load -> Transform	Carregar os dados primeiro e transformar

Fluxo	Ordem	Ideia central
		depois dentro do warehouse.

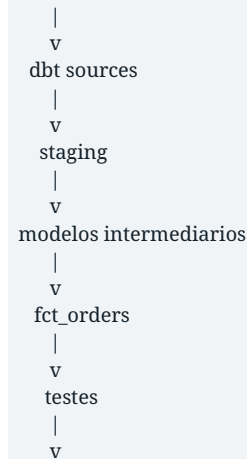
Motivação da aula: no passado, armazenamento em cloud/data warehouse era caro; transformar antes reduzia o volume carregado. Com plataformas modernas como Snowflake, armazenamento ficou relativamente mais barato e passou a fazer sentido carregar os dados e transformá-los depois.

Neste projeto - Snowflake recebe/expõe os dados; dbt executa as transformações dentro do warehouse; Airflow orquestra a execução.

2. Arquitetura e ferramentas

- Snowflake - data warehouse e ambiente de execução SQL.
- TPC-H / SNOWFLAKE_SAMPLE_DATA.TPCH_SF1 - dataset de exemplo usado na aula.
- dbt Core - transformação, modelagem, macros, testes e organização do projeto.
- dbt_utils - pacote auxiliar usado para criar surrogate keys.
- Airflow - orquestração do pipeline.
- Astronomer Cosmos - integração do projeto dbt com DAGs/Task Groups do Airflow.

SNOWFLAKE_SAMPLE_DATA.TPCH_SF1



3. Preparação do Snowflake

O professor cria o ambiente de trabalho antes de inicializar o dbt. A lógica é: ACCOUNTADMIN cria os objetos principais, concede permissões à DBT_ROLE e depois a DBT_ROLE cria o schema onde os models do dbt serão materializados.

3.1 Sequência usada na aula

```

USE ROLE ACCOUNTADMIN;

CREATE WAREHOUSE DBT_WH WITH WAREHOUSE_SIZE = 'X-SMALL';
CREATE DATABASE IF NOT EXISTS DBT_DB;
CREATE ROLE IF NOT EXISTS DBT_ROLE;

SHOW GRANTS ON WAREHOUSE DBT_WH;

GRANT USAGE ON WAREHOUSE DBT_WH TO ROLE DBT_ROLE;
GRANT ROLE DBT_ROLE TO USER RAELDUTRASNOW;
GRANT ALL ON DATABASE DBT_DB TO ROLE DBT_ROLE;

USE ROLE DBT_ROLE;

CREATE SCHEMA DBT_DB.DBT_SCHEMA;
  
```

- ACCOUNTADMIN funciona como a role administrativa/superusuário do Snowflake.
- Warehouse é o recurso de compute usado para executar queries.
- Database organiza os objetos de dados.

- Role concentra as permissões do dbt.
- Schema é o namespace dentro do DBT_DB onde o dbt vai criar views/tables.

CHECKPOINT - Ao atualizar a árvore de Databases, DBT_DB deve existir e DBT_SCHEMA deve aparecer dentro dele.

3.2 Limpeza mostrada pelo professor

A aula também mostra comandos de DROP para limpar o ambiente e evitar manter recursos desnecessários. Eles não fazem parte do fluxo principal de criação.

```
USE ROLE ACCOUNTADMIN;
DROP WAREHOUSE IF EXISTS DBT_WH;
DROP DATABASE IF EXISTS DBT_DB;
DROP ROLE IF EXISTS DBT_ROLE;
```

Atenção - No Snowflake, "" não é comentário SQL. Para deixar o bloco de DROP guardado sem executar, use -- em cada linha ou /* ... */.

4. Inicializar o dbt Core e conectar ao Snowflake

Depois do Snowflake, o professor inicializa o projeto local com dbt Core.

```
dbt init
```

No seu projeto, as respostas usadas foram:

Campo	Valor
project	data_pipeline
adapter	snowflake
account	<ACCOUNT_IDENTIFIER>
user	RAELDUTRASNOW
authentication	password
role	DBT_ROLE
warehouse	DBT_WH
database	DBT_DB
schema	DBT_SCHEMA
threads	10

```
cd .data_pipeline
dbt debug
```

CHECKPOINT - O resultado correto é: profiles.yml [OK], dbt_project.yml [OK], Connection test [OK connection ok] e "All checks passed!".

Importante - O dbt init cria automaticamente a estrutura do projeto. Você não criou manualmente analyses/, macros/, models/, seeds/, snapshots/, tests/, dbt_project.yml e README.md.

5. Estrutura inicial do projeto dbt

```
data_pipeline/
|-- analyses/
|-- macros/
|-- models/
|   |-- example/    # criado pelo dbt init; a aula remove
|-- seeds/
|-- snapshots/
|-- tests/
|-- dbt_project.yml
`-- README.md
```

- models/ - arquivos SQL que viram views/tables no Snowflake.
- macros/ - lógica reutilizável em Jinja/SQL.
- seeds/ - arquivos estáticos, normalmente CSV, carregados pelo dbt.
- snapshots/ - recurso para acompanhar mudanças/histórico de registros.

- tests/ - testes singulares escritos como queries SQL.
- dbt_project.yml - arquivo central que informa onde estão modelos, testes, seeds, macros e configurações de materialização.

A aula remove os models de exemplo e cria duas áreas dentro de models:

```
models/
|-- staging/
`-- marts/
```

5.1 Configuração do dbt_project.yml

A lógica descrita pelo professor é materializar staging como view e marts como table, usando o warehouse do dbt.

```
models:
  data_pipeline:
    staging:
      +materialized: view
      +snowflake_warehouse: DBT_WH

  marts:
    +materialized: table
    +snowflake_warehouse: DBT_WH
```

Conceito - Staging fica próximo de 1:1 com a fonte e costuma ser leve; por isso a aula usa views. Marts representam saídas analíticas e são materializados como tables.

6. dbt_utils e dbt deps

A aula adiciona o pacote dbt_utils para reutilizar funcionalidades prontas, principalmente a geração de surrogate key.

```
packages:
- package: dbt-labs/dbt_utils
  version: 1.1.1
```

Depois:

```
dbt deps
```

No seu terminal - Você instalou dbt-labs/dbt_utils 1.1.1. O dbt avisou que existe versão mais nova, mas para acompanhar a aula não é necessário atualizar agora.

7. Sources: TPC-H no Snowflake

A aula cria um arquivo YAML para declarar as tabelas de origem. O nome exato do arquivo não importa; o professor usa um arquivo de sources dentro de models/staging.

```
version: 2

sources:
- name: tpch
  database: snowflake_sample_data
  schema: tpch_sf1

tables:
- name: orders
  columns:
    - name: o_orderkey
    tests:
      - unique
      - not_null

- name: lineitem
  columns:
    - name: l_orderkey
    tests:
      - relationships:
          to: source('tpch', 'orders')
          field: o_orderkey
```

- ORDERS: o_orderkey deve ser único e não nulo.
- LINEITEM: l_orderkey deve apontar para uma chave válida em ORDERS.
- Esses são exemplos de generic tests configurados em YAML.

Erro mostrado na aula - YAML é sensível à indentação. O professor corrige um erro causado por indentação/nome incorreto antes de o model rodar.

8. Staging de ORDERS

O primeiro model de staging lê a source ORDERS e renomeia colunas para nomes mais limpos.

```
select
  o_orderkey as order_key,
  o_custkey as customer_key,
  o_orderstatus as status_code,
  o_totalprice as total_price,
  o_orderdate as order_date
from {{ source('tpch', 'orders') }}
```

O ponto importante é a função source():

```
{{ source('tpch', 'orders') }}
```

Ela referencia a fonte declarada no YAML, em vez de escrever o caminho físico diretamente no SQL.

```
dbt run
```

CHECKPOINT - Depois do run, atualize o Snowflake. O model de staging deve aparecer em DBT_DB.DBT_SCHEMA e, pela configuração da aula, como VIEW.

9. Staging de LINEITEM + surrogate key

O segundo staging lê LINEITEM e cria uma chave substituta a partir de order key + line number.

Limite da transcrição - A transcrição não preserva a lista completa de colunas que o professor copia na tela. Ela confirma, porém, o uso de L_ORDERKEY, L_LINENUMBER e da surrogate key; depois também usa extended price e discount percentage.

```
select
  {{ dbt_utils.generate_surrogate_key([
    'l_orderkey',
    'l_linenumber'
  ]) }} as order_item_key,

  l_orderkey as order_key,
  l_linenumber as line_number,
  l_extendedprice as extended_price,
  l_discount as discount_percentage
from {{ source('tpch', 'lineitem') }}
```

- Surrogate key: chave técnica criada pelo modelo, útil para conectar tabelas em modelagem dimensional.
- Na aula, order key + line number identificam de forma estável o item do pedido.
- O professor recebe um aviso de função antiga e troca para o nome novo da função de surrogate key.

```
dbt run -s stg_tpch_line_items
```

A opção -s (select) executa somente o model escolhido.

10. Transformações e conceito de fact table

A partir daqui, a aula sai do staging 1:1 e começa a aplicar lógica de negócio. O professor explica fact table como uma tabela que registra medidas numéricas de um processo de negócio e que se conecta a dimensões por chaves.

Ideia da aula - Staging organiza/renomeia a fonte; modelos intermediários juntam e transformam; o fact model entrega a visão analítica consolidada.

10.1 int_order_items.sql

O model intermediário referencia os dois stagings usando ref() e faz o JOIN por order_key.

```
select
  line_item.order_item_key,
  line_item.order_key,
  line_item.line_number,
  line_item.extended_price,
  line_item.discount_percentage,
  orders.customer_key,
  orders.status_code,
  orders.order_date
from {{ ref('stg_tpch_orders') }} as orders
join {{ ref('stg_tpch_line_items') }} as line_item
  on orders.order_key = line_item.order_key
order by orders.order_date

{{ ref('stg_tpch_orders') }}
{{ ref('stg_tpch_line_items') }}
```

ref() cria dependência entre models e permite ao dbt montar o DAG do projeto.

Erro mostrado na aula - O professor corrige um erro simples de sintaxe/comma e reroda os models. A mensagem do dbt indica qual model/linha precisa ser revisado.

11. Macro de pricing/desconto

O professor cria macros/pricing.sql para centralizar uma regra de negócio reutilizável. A macro recebe extended price e discount percentage e devolve o valor do desconto.

Limite da transcrição - A transcrição descreve os argumentos e o objetivo da macro, mas não preserva exatamente a expressão digitada na tela. O bloco abaixo representa a lógica necessária para acompanhar o restante da aula; compare visualmente com o vídeo no momento em que o professor cria pricing.sql.

```
{% macro discounted_amount(extended_price, discount_percentage) %}
  {{ extended_price }} * {{ discount_percentage }}
{% endmacro %}
```

Uso no model intermediário:

```
{{ discounted_amount(
  'line_item.extended_price',
  'line_item.discount_percentage'
)}} as item_discount_amount

dbt run -s int_order_items
```

- Macro evita repetir a mesma regra em vários models.
- O SQL compilado pelo dbt contém a expressão final gerada pelo Jinja.
- Essa é a ideia de templating/reuso que o professor quer demonstrar.

12. Summary e fct_orders

12.1 int_order_items_summary.sql

A aula agrega os itens por pedido usando GROUP BY, somando extended price e discount amount.

```
select
  order_key,
  sum(extended_price) as gross_item_sales_amount,
  sum(item_discount_amount) as item_discount_amount
from {{ ref('int_order_items') }}
group by order_key
```

12.2 fct_orders.sql

O fact model junta os dados do pedido com o resumo calculado dos itens.

```
select
  orders.*,
  order_item_summary.gross_item_sales_amount,
  order_item_summary.item_discount_amount
from {{ ref('stg_tpch_orders') }} as orders
join {{ ref('int_order_items_summary') }} as order_item_summary
  on orders.order_key = order_item_summary.order_key
order by orders.order_date

dbt run
```

CHECKPOINT - No Snowflake, atualize DBT_SCHEMA e confirme a criação do fact model. A aula mostra FCT_ORDERS materializada e conectada ao restante do fluxo.

13. Testes no dbt

A aula diferencia dois tipos: generic tests e singular tests.

Tipo	Onde	Como funciona
Generic	YAML em models/	Teste parametrizado: unique, not_null, relationships, accepted_values.
Singular	tests/*.sql	Query que retorna linhas inválidas. Se retornar linha, o teste falha.

13.1 Generic tests

```
version: 2

models:
  - name: fct_orders
    columns:
      - name: order_key
        tests:
          - unique
          - not_null
          - relationships:
              to: ref('stg_tpch_orders')
              field: order_key
              severity: warn

      - name: status_code
        tests:
          - accepted_values:
              values: ['P', 'O', 'F']

dbt test
```

13.2 Singular test - desconto

A lógica da aula é retornar registros com desconto inválido. O professor também inverte a condição de propósito para demonstrar uma falha com muitas linhas.

```
select *
from {{ ref('fct_orders') }}
where item_discount_amount > 0
```

13.3 Singular test - data válida

```
select *
from {{ ref('fct_orders') }}
where cast(order_date as date) > current_date
  or cast(order_date as date) < '1990-01-01'

dbt test
```

Regra mental - Singular test = query de erros. Zero linhas = passou. Uma ou mais linhas = falhou.

14. Deploy/orquestração com Airflow + Astronomer Cosmos

Com o projeto dbt pronto, a aula passa para Airflow. O professor escolhe Airflow, mas menciona Prefect e Dagster como alternativas. Cosmos é usado para transformar o projeto dbt em tarefas/Task Groups do Airflow.

14.1 Astro CLI e projeto Airflow

No ambiente do professor:

```
brew install astro  
  
mkdir dbt_dag  
cd dbt_dag  
astro dev init
```

Windows - brew install astro é específico do ambiente do professor. No seu Windows, instale o Astro CLI pelo método oficial para Windows quando chegar nessa parte. O restante do fluxo da aula continua igual.

14.2 Dockerfile

A aula adiciona uma instalação de dbt-snowflake ao Dockerfile do projeto Astro. A transcrição não preserva a linha exata mostrada na tela; copie essa linha do vídeo neste ponto.

14.3 requirements.txt do Airflow

A transcrição confirma estas duas dependências:

```
astronomer-cosmos  
apache-airflow-providers-snowflake
```

Depois:

```
astro dev start
```

- Airflow local abre em `http://localhost:8080`.
- Na aula, usuário e senha locais são `admin / admin`.

15. Integrar o projeto dbt ao DAG

O professor copia a pasta do projeto `data_pipeline` para dentro da estrutura de dags do projeto Airflow e cria o arquivo do DAG usando Cosmos.

Limite da transcrição - O código completo do DAG estava na tela do professor, mas não foi preservado na transcrição textual. Para acompanhar fielmente, copie o arquivo do vídeo neste ponto. Abaixo estão os elementos que a transcrição confirma.

- Imports de `datetime` e `os`.
- Imports da biblioteca `cosmos`.
- Connection ID do Snowflake (`snowflake_conn`).
- Profile/config apontando para `DBT_DB` e `DBT_SCHEMA`.
- Project path apontando para a pasta do projeto dbt copiada para dags.
- DAG ID chamado `dbt_dag`.
- Agendamento diário.
- Cosmos lê as dependências do dbt e cria o grafo de tarefas.

```
dags/  
|-- dbt/  
|   |-- data_pipeline/  
|       |-- dbt_project.yml  
|       |-- models/  
|       |-- macros/  
|       |-- ...  
|-- dbt_dag.py
```

16. Criar a conexão Snowflake no Airflow

Na interface do Airflow:

- Admin -> Connections -> Add a connection.
- Connection ID: snowflake_conn.
- Connection type: Snowflake.
- Preencher account, user/password, warehouse, database e role.
- Warehouse: DBT_WH.
- Database: DBT_DB.
- Role: DBT_ROLE.
- Salvar.

Erro mostrado na aula - A primeira execução falha porque a conexão não tinha username/password preenchidos. O professor volta em Admin -> Connections, completa as credenciais, salva e dispara o DAG novamente.

17. Executar o DAG e ler o Graph

Antes de disparar, a aula abre a visualização Graph. Cosmos transforma a dependência dos models dbt em um grafo visível no Airflow.

```
stg_tpch_orders -----\
                        -> int_order_items -> int_order_items_summary -> fct_orders
stg_tpch_line_items -----/

+ testes associados
```

- Trigger DAG para iniciar manualmente.
- Abra a execução e acompanhe o status de cada task.
- Em caso de falha, abra Logs da task.
- Após corrigir a conexão Snowflake, a aula executa com sucesso.
- Nos logs é possível ver o dbt sendo executado dentro das tasks.

Observação do professor - XCom não é a melhor opção para passar grandes volumes de dados entre tasks; existe limite/praticidade de uso. O pipeline deve mover dados via warehouse/storage, não por XCom.

18. Estrutura final do projeto da aula

```
data_pipeline/
|-- macros/
|   |-- pricing.sql
|-- models/
|   |-- staging/
|   |   |-- tpch_sources.yml
|   |   |-- stg_tpch_orders.sql
|   |   |-- stg_tpch_line_items.sql
|   |-- marts/
|   |   |-- int_order_items.sql
|   |   |-- int_order_items_summary.sql
|   |   |-- fct_orders.sql
|   |-- generic_tests.yml
|-- tests/
|   |-- fct_orders_discount.sql
|   |-- fct_orders_date_valid.sql
|-- dbt_project.yml
|-- packages.yml
|-- package-lock.yml
```

No Snowflake, o resultado lógico é:

```

SNOWFLAKE_SAMPLE_DATA.TPCH_SF1
|
| source()
v
DBT_DB.DBT_SCHEMA
|-- STG_TPCH_ORDERS      (view)
|-- STG_TPCH_LINE_ITEMS (view)
|-- INT_ORDER_ITEMS     (table, pela config de marts)
|-- INT_ORDER_ITEMS_SUMMARY (table)
`-- FCT_ORDERS          (table)

```

19. Comandos para memorizar

Estes são os comandos dbt que realmente aparecem no fluxo da aula e valem decorar:

```

dbt init
# cria/inicializa o projeto e o profile

dbt debug
# valida arquivos de configuracao e conexao

dbt deps
# instala pacotes do packages.yml

dbt run
# executa os modelos

dbt run -s nome_do_modelo
# executa apenas um modelo selecionado

dbt test
# executa os testes

```

Comandos Astro/Airflow que aparecem depois:

```

astro dev init
astro dev start

```

20. Checklist para acompanhar o professor

- [] Snowflake: warehouse, database, role e schema criados.
- [] dbt init concluído e projeto data_pipeline criado automaticamente.
- [] dbt debug termina em All checks passed.
- [] models/example removido; staging e marts criados.
- [] dbt_project.yml: staging=view; marts=table.
- [] packages.yml com dbt_utils; dbt deps concluído.
- [] Source tpch aponta para SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.
- [] ORDERS com unique/not_null em order key.
- [] LINEITEM com relationships para ORDERS.
- [] stg_tpch_orders criado e visível no Snowflake.
- [] stg_tpch_line_items criado com surrogate key.
- [] int_order_items faz JOIN usando ref().
- [] pricing.sql criado e macro aplicada.
- [] int_order_items_summary criado com GROUP BY.
- [] fct_orders criado.
- [] Generic tests passam.
- [] Singular tests passam.
- [] Projeto Astro/Airflow inicializado.
- [] Cosmos + Snowflake provider instalados.
- [] snowflake_conn configurada no Airflow.
- [] dbt_dag aparece no Graph e executa com sucesso.

Ao terminar a aula - A base aprendida é: Snowflake RBAC + dbt sources/staging/marts + ref/source + surrogate key + macros + fact table + generic/singular tests + orquestração no Airflow/Cosmos.

21. Troubleshooting rápido

Sintoma	O que conferir
Warehouse already exists	Você está rerodando CREATE WAREHOUSE sem IF NOT EXISTS. O objeto já existe.
Database 'DBT_DB' does not exist or not authorized	Confirme que DBT_DB existe e que DBT_ROLE recebeu grant no database antes de criar o schema.
YAML compilation/config error	Revise indentação e nomes (name vs names etc.).
dbt_project.yml not found	Você está rodando dbt fora da pasta que contém dbt_project.yml.
Incorrect username or password	Valide o user/login exato do Snowflake e o profiles.yml.
dbt deps says update available	É aviso, não falha. Para acompanhar a aula, pode manter a versão instalada.
Singular test falha com muitas linhas	A query do teste está retornando registros. Teste singular deve retornar zero linhas quando os dados estão válidos.
Airflow DAG falha na Snowflake Connection	Revise a conexão snowflake_conn; na aula faltavam user/password na primeira tentativa.

Fonte e escopo

Este material foi produzido a partir da transcrição fornecida da aula. A transcrição contém a sequência completa dos conceitos e ações, mas não captura perfeitamente todo código que o professor exhibe apenas na tela. Nesses pontos, o guia marca explicitamente a limitação em vez de fingir que a transcrição preservou o conteúdo exato.

O guia também usa os nomes já adotados no seu ambiente (DBT_WH, DBT_DB, DBT_ROLE, DBT_SCHEMA, projeto data_pipeline) para reduzir atrito durante o acompanhamento. Senhas e identificadores sensíveis não são incluídos.